IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

ETAPA B · CONSTRUIR · SPRINT 8 DE 17

Sprint 8 · Árboles, ensembles y SVM

¿cuál es el mejor modelo y cuánto nos cuesta mantenerlo?

FECHAS
10–11 de noviembre

HORAS
12 h (3 h teoría · 4 h práctica guiada · 5 h PBL)

CONTENIDOS
M3 UD 3.6 (árboles de decisión), UD 3.7 (Random Forest, Gradient Boosting, XGBoost y LightGBM) y UD 3.8 (SVM), más ajuste de hiperparámetros (`GridSearchCV`, `RandomizedSearchCV`).

EVALUACIÓN
test de M3 (primera parte) y reto PBL «Duelo de modelos».

Qué aprenderás

1. Entrenar un **árbol de decisión**, detectar su sobreajuste, podarlo y leer sus reglas y su importancia de variables.
2. Explicar la diferencia entre **bosque aleatorio** (promediar árboles) y **boosting** (corregir errores en cadena), y usar XGBoost y LightGBM con parada temprana.
3. Ajustar una **SVM** (`C`, `gamma`, kernel) y saber cuándo su coste la descarta.
4. Buscar hiperparámetros con **`GridSearchCV`** y **`RandomizedSearchCV`** sin engañarte con la nota de validación.
5. **Elegir un modelo campeón** justificando rendimiento, explicabilidad y coste, y documentarlo en una *model card*.

Requisitos previos

- Sprint 6 y 7: pipelines, validación cruzada, métricas de clasificación (AUC, precisión, recall), umbral y regresión logística como referencia.
- Datos: `datos/reservas_turisdata.csv` (objetivo `cancelada`). Los notebooks lo descargan solos si no lo encuentran.
- Librerías: scikit-learn, xgboost y lightgbm (vienen instaladas en Colab; en local: `pip install xgboost lightgbm`).

Prácticas

Notebook	Tema	Tiempo	Colab
`S08_01_arboles_decision`	Árbol de decisión: reglas, sobreajuste, poda, importancia	45 min	Abrir en Colab
`S08_02_random_forest`	Random Forest: bagging, OOB, importancia por permutación	45 min	Abrir en Colab
`S08_03_boosting_xgboost_lightgbm`	Gradient Boosting, XGBoost, LightGBM y parada temprana	55 min	Abrir en Colab
`S08_04_svm`	SVM: margen, kernels, `C`, `gamma`, escalado y coste	40 min	Abrir en Colab
`S08_05_busqueda_hiperparametros`	`GridSearchCV`, `RandomizedSearchCV`, `cv_results_`, test final	45 min	Abrir en Colab
`S08_06_leaderboard_plantilla`	**Herramienta del reto**: leaderboard para rellenar	dentro del reto	Abrir en Colab

Las cinco primeras son las prácticas guiadas; la sexta es la plantilla que usaréis en el reto. Los ejercicios se autocorrigen con `assert`.

RETO Reto PBL · «Duelo de modelos» (unas 4 h, en equipo)

En pocas palabras (léelo primero)

Qué os pide el cliente. TurisData ya sabe que se pueden predecir las cancelaciones (lo visteis en el Sprint 7). Ahora la pregunta es: **¿qué modelo ponemos en producción?** El gerente duda: *«¿el más preciso? ¿el que yo pueda entender? ¿el más barato de mantener?»*. Vuestro equipo tiene que **enfrentar varios modelos** (árboles, bosques, boosting y SVM), compararlos con las mismas reglas y **elegir un campeón con argumentos**.

La idea clave: en la vida real **el modelo más preciso no siempre es el mejor**. Un modelo un poco peor, pero que se entiende y se mantiene fácilmente, puede ser la mejor decisión. Aprenderéis a decidir con **tres criterios**: rendimiento, explicabilidad y coste.

Qué vais a hacer, en seis pasos:

1. **Acordar los pesos** de los tres criterios con la «voz del cliente» (¿qué importa más?).
2. **Entrenar y registrar** al menos 6 modelos en el *leaderboard* (tabla de resultados), todos medidos igual.
3. **Buscar los mejores ajustes** (hiperparámetros) para al menos dos familias de modelos.
4. **Elegir el campeón** y comprobar si la elección cambia al variar los pesos.
5. **Evaluar el campeón en el test, una sola vez.**
6. **Escribir la *model card*** (la «ficha técnica» del modelo) y el README.

Qué vais a entregar: `leaderboard_S08.csv` (plantilla), `model_card.md` (plantilla), el cuaderno `reto_S08.ipynb` y `README.md`.

Qué vais a aprender haciéndolo: a comparar modelos de forma **justa**, a **ajustar hiperparámetros sin hacer trampas**, y a **documentar un modelo** como se hace en las empresas.

Cuánto tiempo: unas 4 horas en equipo entre el planning, el reto y la entrega (10 y 11/11).

Palabras que van a aparecer en este reto

Palabra	Qué significa, en sencillo
Leaderboard	Tabla de clasificación de los modelos, con sus resultados
Árbol de decisión	Modelo que decide con una cadena de preguntas sí/no. Fácil de explicar
Bosque aleatorio (random forest)	Muchos árboles que «votan». Más preciso, menos fácil de explicar
Boosting	Árboles que se construyen uno tras otro, corrigiendo los errores del anterior. Muy potente
SVM	Modelo que busca la mejor «frontera» entre las clases
Hiperparámetro	Un ajuste que hacéis vosotros **antes** de entrenar (profundidad del árbol, número de árboles…)
Búsqueda en rejilla / aleatoria	Probar combinaciones de hiperparámetros de forma ordenada o al azar
Validación cruzada (CV)	Evaluar varias veces con particiones distintas para tener una medida fiable
Folds	Los trozos en que se divide el train en la validación cruzada
AUC con incertidumbre	El AUC acompañado de cuánto varía de un fold a otro (media ± desviación)
Explicabilidad	Poder decir **por qué** el modelo toma una decisión
Model card	Documento de 1–2 páginas que describe un modelo: usos, datos, métricas, límites, riesgos
Deriva (drift)	Cuando el comportamiento de los clientes cambia con el tiempo y el modelo deja de funcionar bien

Contexto del cliente. TurisData ya sabe que puede predecir cancelaciones (Sprint 7). Ahora quiere saber **qué modelo poner en producción**. El gerente pregunta: «¿el más preciso? ¿el que puedo entender? ¿el más barato de mantener?». Vuestro equipo compite consigo mismo: tenéis que enfrentar **árboles, bosques, boosting y SVM**, y **elegir un campeón** con argumentos.

Qué se entrega.

1. **Leaderboard**: `leaderboard_S08.csv` con al menos **6 modelos** (baseline y logística incluidos) medidos igual. Se rellena con el notebook `S08_06_leaderboard_plantilla` (plantilla del CSV: `plantillas/leaderboard_plantilla.csv`).
2. **Model card** del campeón: `model_card.md` siguiendo `plantillas/model_card_plantilla.md` (1–2 páginas).
3. `reto_S08.ipynb` con la comparación reproducible y la búsqueda de hiperparámetros de, al menos, **dos** familias.
4. `README.md` para reproducirlo (versiones, semilla, orden de ejecución).

Restricciones.

- Mismos datos, mismos folds y misma métrica para todos los modelos (ya lo garantiza la plantilla).
- La búsqueda de hiperparámetros se hace **solo sobre train**. **El test se usa una vez**, únicamente con el campeón.
- Máximo 8 modelos en el leaderboard; tiempo máximo de cada búsqueda: 5 minutos en Colab.
- Solo librerías del curso. Semilla fija (`SEMILLA = 42`).
- El campeón se justifica con **tres** criterios: **rendimiento** (AUC con incertidumbre), **explicabilidad** (¿puede el gerente entender por qué?) y **coste** (tiempo, tamaño, mantenimiento).

Criterios de evaluación (0–4 cada uno; misma rúbrica de todos los retos):

Criterio	1 · Insuficiente	2 · Suficiente	3 · Bueno	4 · Excelente
Corrección técnica	Errores que invalidan el resultado	Correcto con fallos menores	Correcto y bien justificado	Correcto, justificado y con alternativas comparadas
Reproducibilidad	No se puede volver a ejecutar	Se ejecuta con ayuda	Se ejecuta desde el README	Además incluye datos o script y versiones fijas
Análisis y comunicación al cliente	Solo código o cifras sin interpretar	Interpretación básica	Conclusiones claras para el cliente	Recomendación accionable con límites explicados
IA responsable	No se considera	Menciones genéricas	Riesgos específicos identificados	Medidas aplicadas y evaluadas

Pistas de IA responsable (en la model card): usos previstos y **no** previstos, qué variables sensibles hay (país de origen), qué pasa si el comportamiento de los clientes cambia (deriva), cómo se vigilará el modelo y quién revisa las decisiones.

Pasos sugeridos.

- **Martes 10/11:** *planning* (25 min); acordar los **pesos** de rendimiento/explicabilidad/coste con la «voz del cliente»; registrar baseline, logística, árbol podado y bosque en el leaderboard (40 min).
- **Miércoles 11/11, por la mañana:** boosting y SVM en las prácticas; después, búsqueda de hiperparámetros (rejilla o aleatoria) para las dos familias más prometedoras y actualización del leaderboard (35 min).
- **Cierre del miércoles:** sensibilidad de los pesos, elección del campeón, evaluación en test **una vez**, *model card* y README (105 min); autoevaluación y coevaluación; **review + retrospectiva** (última hora).

Guía paso a paso para principiantes

Paso 1 · Planning y pesos de los criterios (martes 10/11, 25 minutos)

1. Roles y tareas en el tablero.
2. **Decidid los pesos** de los tres criterios, con una justificación («el gerente necesita entender por qué se avisa»):

Criterio	Qué mide	Peso (ejemplo)
Rendimiento	AUC con incertidumbre (media ± desviación en la CV)	50 %
Explicabilidad	¿Puede el gerente entender por qué? (Alta, media o baja)	30 %
Coste	Tiempo de entrenamiento, tamaño, mantenimiento	20 %

Los pesos son vuestros; lo importante es que estén **justificados** y que luego probéis cómo cambia la decisión si cambian.

Paso 2 · Primeros modelos en el leaderboard (martes 10/11, 40 minutos)

1. Abrid `S08_06_leaderboard_plantilla` (el cuaderno ya garantiza **mismos datos, mismos folds y misma métrica** para todos).
2. Registrad el **modelo de referencia (baseline)**, la **regresión logística**, un **árbol podado** y un **bosque**.
3. Cada fila del leaderboard: modelo, familia, AUC medio ± desviación, tiempo de entrenamiento y nota de explicabilidad.

Paso 3 · Boosting, SVM y búsqueda de hiperparámetros (miércoles 11/11, 35 minutos; los modelos salen de las prácticas)

1. Añadid **boosting** y **SVM**.
2. Para las **dos familias más prometedoras**, haced una búsqueda de hiperparámetros (rejilla o aleatoria) **solo sobre train**, con validación cruzada. Cada búsqueda no debe pasar de **5 minutos en Colab**.
3. Actualizad el leaderboard con los mejores modelos. Máximo **8 modelos** en total.

Paso 4 · Elegir el campeón (miércoles 11/11, 60 minutos)

1. Calculad la **puntuación ponderada** de cada modelo con los pesos del paso 1.
2. **Análisis de sensibilidad:** cambiad los pesos (por ejemplo, 30/50/20 o 70/10/20) y mirad si **cambia el campeón**. Si cambia fácilmente, decidlo en el informe: la decisión depende de lo que el cliente valore.
3. **Evaluad el campeón en el test, una sola vez.** Ese número va en la model card. No se evalúan los demás modelos en test.

Paso 5 · Model card y README (miércoles 11/11, 45 minutos)

Seguid `model_card_plantilla.md`. Incluye:

1. **Usos previstos y no previstos.**
2. **Datos** de entrenamiento y sus límites (sintéticos).
3. **Métricas** (con incertidumbre) y rendimiento en test.
4. **Limitaciones** y variables sensibles (país de origen).
5. **Deriva y mantenimiento:** qué mediríais en producción para saber que el modelo se degrada.
6. **IA responsable:** quién revisa las decisiones del modelo.

README: versiones, `SEMILLA = 42` y orden de ejecución.

Cómo usar la IA en este reto

Para qué	Ejemplo de petición	Qué tenéis que comprobar
Entender un modelo	«Explícame cómo funciona un bosque aleatorio con una analogía de votación de expertos, sin fórmulas»	Que podáis explicarlo al gerente
Entender un hiperparámetro	«¿Qué hace `max_depth` en un árbol? ¿Qué pasa si es muy grande?»	Que probáis valores y vea lo que ocurre (sobreajuste)
Redactar la model card	«Ayúdame a redactar la sección de limitaciones. Estos son los datos: *[resumen]*»	Que **todo** lo que escriba coincide con vuestros resultados

Para qué	Ejemplo de petición	Qué tenéis que comprobar
Preparar la defensa	«Hazme 5 preguntas difíciles que un gerente podría hacer sobre mi elección de campeón»	Que preparéis la respuesta con **vuestros** datos

Reglas del reto:

- **El test se usa una sola vez**, con el campeón. Probar varios modelos en el test y quedarse con el mejor es hacer trampa.
- **La IA no decide el campeón:** lo decide el cálculo con vuestros pesos.
- Declarad dónde habéis usado IA en el README.

Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Comparar modelos con particiones distintas	Cada uno prueba «por su lado»	Usad el cuaderno plantilla: mismos folds para todos
Ajustar hiperparámetros mirando el test	«Para ver si mejora»	Solo con **train y validación cruzada**
Elegir el modelo por el AUC más alto sin ver la desviación	«0,81 > 0,80»	Si los intervalos se solapan, la diferencia puede ser ruido
Model card genérica	Se copia el modelo de ejemplo	Cada apartado con **vuestros** datos y riesgos
Olvidar la explicabilidad y el coste	Solo se mira la precisión	Puntuadlos y ponderadlos
Búsquedas de horas	«Más combinaciones»	Máximo 5 minutos por búsqueda

Un buen resultado se parece a...

> *«Campeón: bosque aleatorio de 200 árboles (AUC 0,78 ± 0,02). No ganó el boosting (AUC 0,79 ± 0,02) porque la diferencia es menor que la incertidumbre, entrena 6 veces más lento y es más difícil de explicar. Si el cliente valorara solo el rendimiento, elegiríamos el boosting.»* (Cifras de ejemplo.)

Fijaos en que se explica **por qué no ganó otro** y **qué cambiaría la elección**.

Antes de entregar: autoevaluación rápida

Pregunta	Sí / No
¿El leaderboard tiene ≥ 6 modelos de al menos 4 familias?	
¿Hay búsqueda de hiperparámetros con CV sobre train para al menos dos familias?	
¿El campeón se justifica con los tres criterios y se probó la sensibilidad de los pesos?	
¿El AUC de test se calculó una sola vez, solo para el campeón?	
¿La model card está completa (usos, datos, métricas, limitaciones, IA responsable, mantenimiento)?	
¿El cuaderno se ejecuta entero y el README tiene versiones?	
¿Está subido a Git, con autoevaluación y coevaluación?	

Si os atascáis

- **No sabemos qué peso poner:** pensad en lo que diría el gerente. Si prioriza entender, sube la explicabilidad.
- **La búsqueda tarda demasiado:** reducid el número de combinaciones o usad búsqueda aleatoria con pocas iteraciones.

- **Dos modelos empatan:** es un resultado válido. Elegid el más simple y explicadlo.
- Más ayuda: Cómo se trabaja un reto.

Qué se enseña en la review (5 min por equipo): el leaderboard (tabla o gráfico), el campeón y **por qué no ganó otro**, y una frase sobre qué cambiaría la elección si el cliente valorara otra cosa.

Preguntas de la retrospectiva: ¿nos sorprendió que un modelo sencillo compitiera con uno complejo? ¿Qué criterio pesó más en nuestra decisión? ¿Qué mediríamos en producción para saber que el modelo se degrada?

Definition of Done

- [ ] `leaderboard_S08.csv` con ≥ 6 modelos de al menos 4 familias (árbol, ensemble, SVM y lineal/baseline).
- [ ] Búsqueda de hiperparámetros con validación cruzada sobre train para al menos dos familias.
- [ ] Campeón elegido con los tres criterios y análisis de sensibilidad de los pesos.
- [ ] AUC de test calculado **una sola vez**, solo para el campeón.
- [ ] `model_card.md` completa (usos, datos, métricas, limitaciones, IA responsable, mantenimiento).
- [ ] Notebook ejecutable de principio a fin y README con versiones.
- [ ] Subido a Git; autoevaluación y coevaluación rellenadas.

Recursos

- Documentación de scikit-learn: *Ensemble methods*, *Decision Trees* y *Support Vector Machines*.
- Documentación de XGBoost y de LightGBM: *Parameters Tuning* (guía de ajuste de hiperparámetros).
- «Model Cards for Model Reporting» (Mitchell et al., 2019): artículo que da origen a las model cards.
- Guía de scikit-learn: *Tuning the hyper-parameters of an estimator*.
- Tu propio notebook `S07_03` (umbral y coste) y el informe del Sprint 7: la recomendación de negocio sigue en pie.